

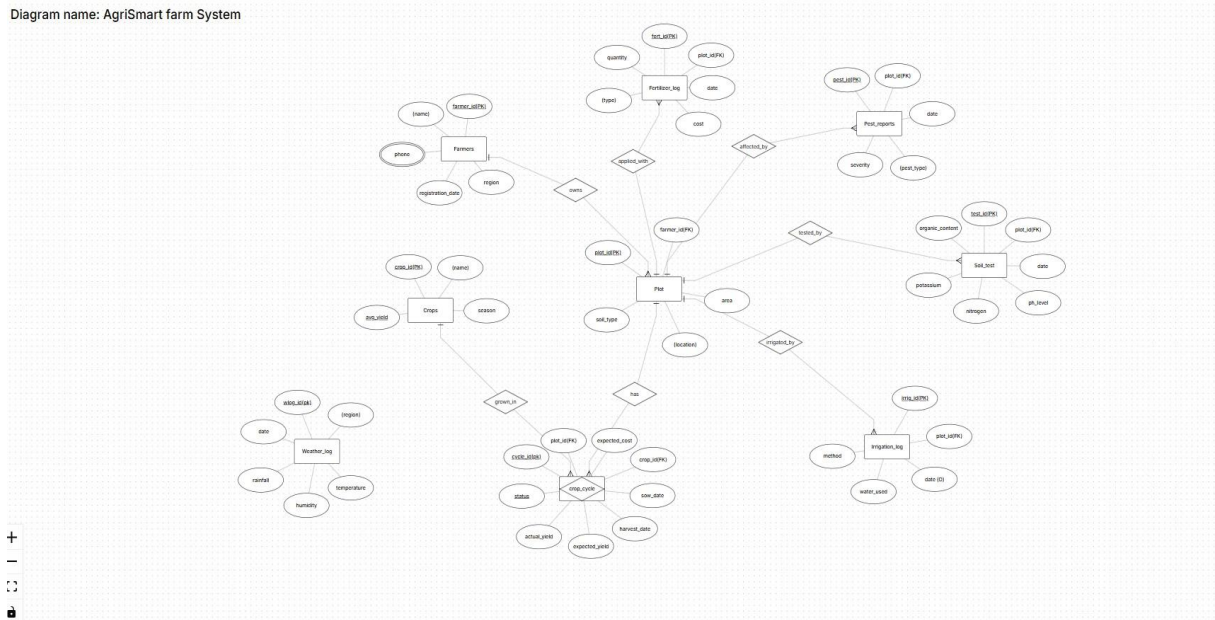
AGRISMART FARM CROP AND YIELD ANALYSIS SYSTEM

Case study : 55

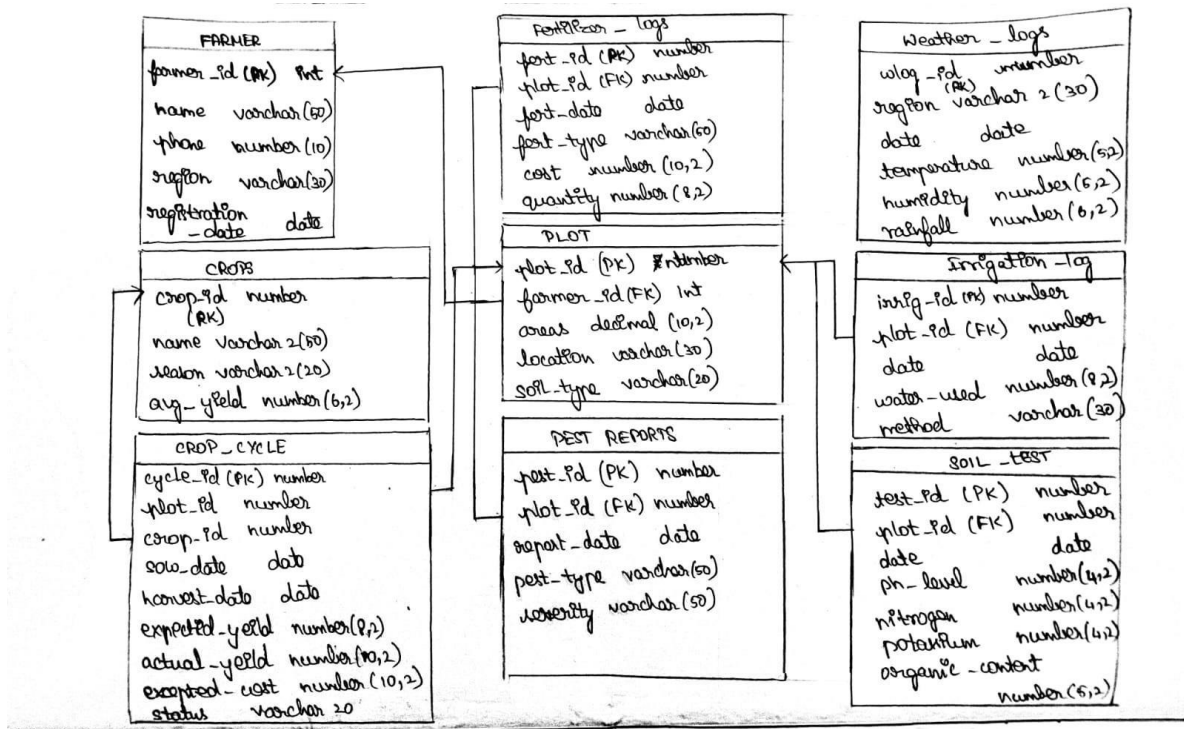
R.Pavithra |717824E237

1. ER DIAGRAM:

Diagram name: AgriSmart farm System



CLASS DIAGRAM:



NORMALIZATION

1NF (First Normal Form)

- Initially, farmer, plot, crop, soil test, irrigation, fertilizer, pest, and weather details were stored together, causing repeating groups and duplicate information.
- To achieve 1NF, the data was separated into individual tables such as Farmers, Plot, Crops, Crop_Cycle, SoilTest, Irrigation_Log, Fertilizer_Logs, Pest_Reports, and Weather_Logs.
- Thus, all attributes contain atomic values and there are no repeating groups.

2NF (Second Normal Form)

- Initially, combining farmer, plot, and crop information in a single table created partial dependencies.
- To achieve 2NF, separate tables were created for Farmers, Plot, Crops, Crop_Cycle, SoilTest, Irrigation_Log, Fertilizer_Logs, Pest_Reports, and Weather_Logs, each having its own primary key.
- Foreign keys were used to establish relationships between tables.
- Hence, all non-key attributes are fully dependent on their respective primary keys.

3NF (Third Normal Form)

- Initially, transitive dependencies existed among farmer, plot, crop, and agricultural activity information.
- To achieve 3NF, redundant data was removed by maintaining separate tables and connecting them using foreign keys.
- For example, crop information is stored only in the **Crops** table, while crop cultivation details are maintained in the **Crop_Cycle** table.
- Thus, transitive dependencies were eliminated, reducing redundancy and improving data integrity.

BCNF (Boyce-Codd Normal Form)

- All tables satisfy the requirements of **BCNF** because every determinant is a candidate key.
- Each table has a unique primary key that determines all other attributes.
- Crop cycles are maintained separately from plots, allowing multiple crops to be cultivated on the same plot across different seasons.
- Therefore, the database design eliminates update anomalies and ensures efficient data management.

CREATE TABLES

Farmer table

```
CREATE TABLE farmers( farmer_id INT  
PRIMARY KEY, farmer_name  
VARCHAR(50), phone VARCHAR(10),  
region VARCHAR(30),  
registration_date DATE);
```

```
3  
4 CREATE TABLE farmers ( farmer_id INT PRIMARY KEY,  
5 farmer_name VARCHAR(50), phone VARCHAR(10),  
6 region VARCHAR(30), registration_date DATE);  
7  
8  
9
```

Query result **Script output** DBMS output Explain Plan SQL



SQL> drop table farmers

Table FARMERS dropped.

Elapsed: 00:00:00.045

Plot table

```
CREATE TABLE plot ( plot_id INT PRIMARY KEY, farmer_id INT,  
areas DECIMAL(10,2), location VARCHAR2(30), soil_type  
VARCHAR2(50), FOREIGN KEY (farmer_id)  
REFERENCES farmers(farmer_id));
```

```
8 CREATE TABLE plots ( plot_id INT PRIMARY KEY, farmer_id INT, areas DECIMAL(10,2),  
9 location VARCHAR2(30), soil_type VARCHAR2(50),  
10 FOREIGN KEY (farmer_id) REFERENCES farmer(farmer_id));  
11  
12  
13  
..
```

Query result **Script output** DBMS output Explain Plan SQL history



SQL> CREATE TABLE plots (plot_id INT PRIMARY KEY, farmer_id INT, areas DECIMAL(10,2),
location VARCHAR2(30), soil_type VARCHAR2(50),
FOREIGN KEY (farmer_id) REFERENCES farmer(farmer_id))

Table PLOTS created.

Elapsed: 00:00:00.027

Soils_test table

```
CREATE TABLE soiltest (test_id NUMBER PRIMARY KEY,plot_id NUMBER,  
test_date DATE,ph_level NUMBER(4,2) CHECK(ph_level  
BETWEEN 0 AND 14), nitrogen  
NUMBER(5,2),potassium NUMBER(4,1),organic_content  
NUMBER(5,2),CONSTRAINT fk_soils_plot FOREIGN  
KEY (plot_id)REFERENCES plots(plot_id));
```

```
63 CREATE TABLE soiltest (  
64     test_id NUMBER PRIMARY KEY,plot_id NUMBER,  
65     test_date DATE,ph_level NUMBER(4,2) CHECK (ph_level BETWEEN 0 AND 14),  
66     nitrogen NUMBER(5,2),potassium NUMBER(4,1),  
67     organic_content NUMBER(5,2),CONSTRAINT fk_soils_plot  
68     FOREIGN KEY (plot_id)REFERENCES plots(plot_id));
```

Query result **Script output** DBMS output Explain Plan SQL history



```
SQL> CREATE TABLE soiltest (  
      test_id NUMBER PRIMARY KEY,  
      plot_id NUMBER,  
      test_date DATE,...  
Show more...
```

Table SOILTEST created.

Elapsed: 00:00:00.038

Crop table

```
CREATE TABLE crops ( crop_id NUMBER PRIMARY KEY, name VARCHAR2(50),  
season VARCHAR2(20), avg_yield NUMBER(6,2));
```

```
14 CREATE TABLE crops ( crop_id NUMBER PRIMARY KEY, name VARCHAR2(50),  
15 season VARCHAR2(20), avg_yield NUMBER(6,2));  
16
```

Query result **Script output** DBMS output Explain Plan SQL history



```
SQL> CREATE TABLE crops ( crop_id NUMBER PRIMARY KEY, name VARCHAR2(50),  
      season VARCHAR2(20), avg_yield NUMBER(6,2))
```

Table CROPS created.

Elapsed: 00:00:00.028

Crop_cycle table

```
CREATE TABLE crop_cycle ( cycle_id NUMBER PRIMARY KEY,  
plot_id NUMBER, crop_id NUMBER, sow_date DATE, harvest_date DATE,  
expected_yield NUMBER(8,2), actual_yield NUMBER(8,2),CHECK  
(actual_yield >= 0),status VARCHAR2(20), expected_cost NUMBER(10,2),  
FOREIGN KEY (plot_id) REFERENCES plots(plot_id),  
FOREIGN KEY (crop_id) REFERENCES crops(crop_id ) );
```

The screenshot shows the SQL Developer interface with the 'Script output' tab selected. The SQL script for creating the 'crop_cycle' table is displayed in the editor. Below the script, the 'Query result' tab shows the execution output, indicating that the table was successfully created.

```
12  
13 CREATE TABLE crop_cycle (  
14     cycle_id NUMBER PRIMARY KEY,  
15     plot_id NUMBER, crop_id NUMBER,  
16     sow_date DATE, harvest_date DATE,  
17     expected_yield NUMBER(8,2), actual_yield NUMBER(8,2)  
18     CHECK (actual_yield >= 0),  
19     status VARCHAR2(20), expected_cost NUMBER(10,2),  
20     FOREIGN KEY (plot_id) REFERENCES plot(plot_id),  
21     FOREIGN KEY (crop_id) REFERENCES crop(crop_id) );  
22
```

Query result Script output DBMS output Explain Plan

SQL> CREATE TABLE crop_cycle (
cycle_id NUMBER PRIMARY KEY,
plot_id NUMBER, crop_id NUMBER,
sow_date DATE, harvest_date DATE, ...
Show more...

Table CROP_CYCLE created.

Elapsed: 00:00:00.029

Irrigation_log table

```
CREATE TABLE irrigation_log( irrig_id NUMBER PRIMARY KEY, plot_id NUMBER,  
irrig_date DATE, water_used NUMBER(8,2), method VARCHAR2(30), FOREIGN KEY  
(plot_id) REFERENCES plots(plot_id) );
```

The screenshot shows the SQL Developer interface with the 'Script output' tab selected. The SQL script for creating the 'irrigation_log' table is displayed in the editor. Below the script, the 'Query result' tab shows the execution output, indicating that the table was successfully created.

```
137  
138 CREATE TABLE irrigation_log (  
139     irrig_id NUMBER PRIMARY KEY,  
140     plot_id NUMBER, irrig_date DATE,  
141     water_used NUMBER(8,2), method VARCHAR2(30),  
142     CONSTRAINT fk_irrigation_plot FOREIGN KEY  
143     (plot_id) REFERENCES plots(plot_id) );  
144
```

Query result Script output DBMS output Explain Plan SQ

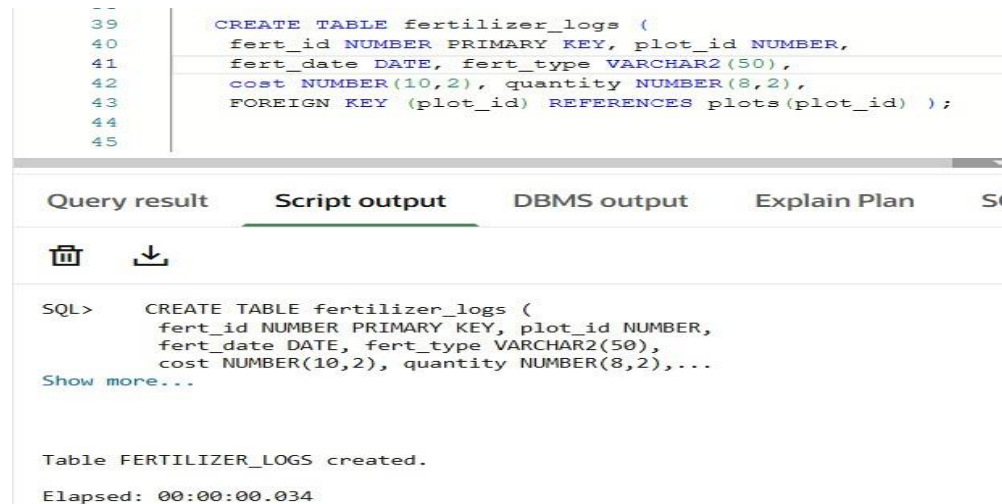
SQL> CREATE TABLE irrigation_log (
irrig_id NUMBER PRIMARY KEY,
plot_id NUMBER, irrig_date DATE,
water_used NUMBER(8,2), method VARCHAR2(30), ...
Show more...

Table IRRIGATION_LOG created.

Elapsed: 00:00:00.032

Fertilizer_logs table

```
CREATE TABLE fertilizer_logs (  
    fert_id NUMBER PRIMARY KEY, plot_id NUMBER,  
    fert_date DATE, fert_type VARCHAR2(50),  
    cost NUMBER(10,2), quantity NUMBER(8,2), FOREIGN KEY (plot_id)  
    REFERENCES plots(plot_id) );
```



The screenshot shows the SQL Developer interface with the 'Script output' tab selected. The SQL script for creating the 'fertilizer_logs' table is displayed in the top pane. The bottom pane shows the execution results, including the message 'Table FERTILIZER_LOGS created.' and the elapsed time '00:00:00.034'.

```
--  
39      CREATE TABLE fertilizer_logs (  
40          fert_id NUMBER PRIMARY KEY, plot_id NUMBER,  
41          fert_date DATE, fert_type VARCHAR2(50),  
42          cost NUMBER(10,2), quantity NUMBER(8,2),  
43          FOREIGN KEY (plot_id) REFERENCES plots(plot_id) );  
44  
45
```

Query result Script output DBMS output Explain Plan SQL

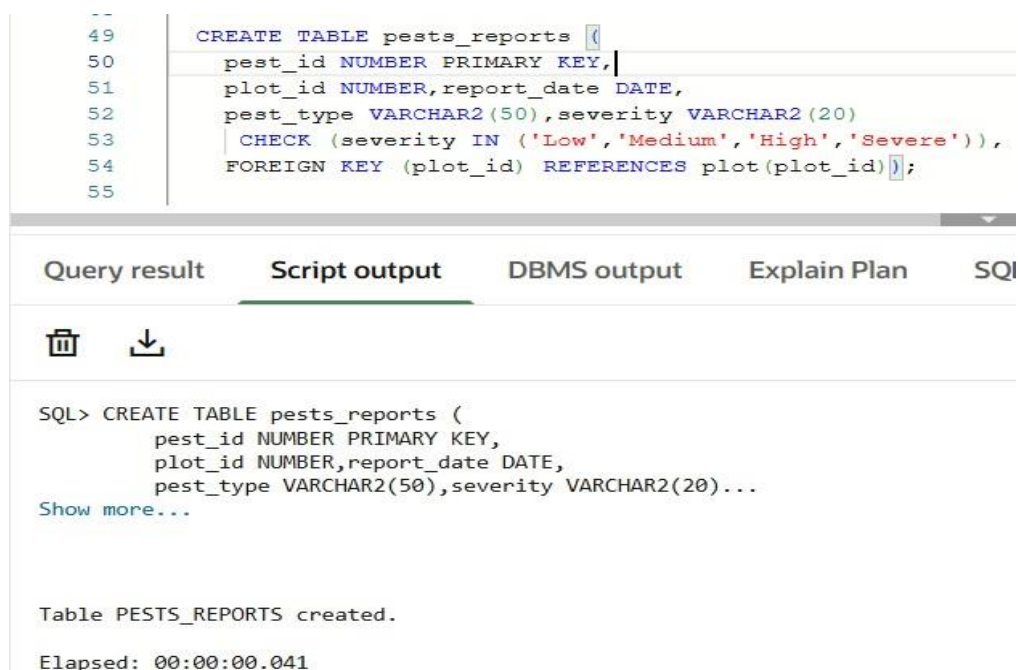
SQL> CREATE TABLE fertilizer_logs (
 fert_id NUMBER PRIMARY KEY, plot_id NUMBER,
 fert_date DATE, fert_type VARCHAR2(50),
 cost NUMBER(10,2), quantity NUMBER(8,2),...
Show more...

Table FERTILIZER_LOGS created.

Elapsed: 00:00:00.034

Pest_reports table

```
CREATE TABLE pest_reports (  
    pest_id NUMBER PRIMARY KEY, plot_id NUMBER, report_date DATE, pest_type  
    VARCHAR2(50), severity VARCHAR2(20), CHECK (severity IN ('Low' , 'Medium',  
    'High' , 'Severe')) FOREIGN KEY (plot_id) REFERENCES plots(plot_id) );
```



The screenshot shows the SQL Developer interface with the 'Script output' tab selected. The SQL script for creating the 'pest_reports' table is displayed in the top pane. The bottom pane shows the execution results, including the message 'Table PESTS_REPORTS created.' and the elapsed time '00:00:00.041'.

```
--  
49      CREATE TABLE pests_reports (  
50          pest_id NUMBER PRIMARY KEY,  
51          plot_id NUMBER, report_date DATE,  
52          pest_type VARCHAR2(50), severity VARCHAR2(20)  
53          CHECK (severity IN ('Low', 'Medium', 'High', 'Severe')),  
54          FOREIGN KEY (plot_id) REFERENCES plots(plot_id) );  
55
```

Query result Script output DBMS output Explain Plan SQL

SQL> CREATE TABLE pests_reports (
 pest_id NUMBER PRIMARY KEY,
 plot_id NUMBER, report_date DATE,
 pest_type VARCHAR2(50), severity VARCHAR2(20)...
Show more...

Table PESTS_REPORTS created.

Elapsed: 00:00:00.041

Weather_logs table

CREATE TABLE weather_logs (wlog_id NUMBER PRIMARY KEY,
region VARCHAR2(30), weather_date DATE, temperature NUMBER(5,2),
humidity NUMBER(5,2), rainfall NUMBER(6,2));

53
54
55
56
57
58

CREATE TABLE weather_logs (
wlog_id NUMBER PRIMARY KEY, region VARCHAR2(30),
weather_date DATE,temperature NUMBER(5,2),
humidity NUMBER(5,2),rainfall NUMBER(6,2));

Query resultScript outputDBMS outputExplain Plan

SQL> CREATE TABLE weather_logs (
wlog_id NUMBER PRIMARY KEY, region VARCHAR2(30),
weather_date DATE,temperature NUMBER(5,2),
humidity NUMBER(5,2),rainfall NUMBER(6,2))

Table WEATHER_LOGS created.

Elapsed: 00:00:00.019

INSERT DATA

Farmer table

INSERT INTO farmers VALUES

(1,'Pavithra','9876543210','Coimbatore',TO_DATE('2025-01-01','YYYY-MM-DD'));

INSERT INTO farmers VALUES

(2,'Raja','9876543211','Erode',TO_DATE('2025-01-02','YYYY-MM-DD'));

INSERT INTO farmers VALUES (3,'Sankar','9876543212','Salem',TO_DATE('2025-01-03','YYYY-MM-DD'));

```
63 INSERT INTO farmers VALUES (1,'Pavithra','9876543210','Coimbatore',TO_DATE('2025-01-01','YYYY-MM-DD'));
64 INSERT INTO farmers VALUES (2,'Raja','9876543211','Erode',TO_DATE('2025-01-02','YYYY-MM-DD'));
65 INSERT INTO farmers VALUES (3,'Sankar','9876543212','Salem',TO_DATE('2025-01-03','YYYY-MM-DD'));
66 INSERT INTO farmers VALUES (4,'Dinesh Kumar','9876543213','Madurai',TO_DATE('2025-01-04','YYYY-MM-DD'));
67 INSERT INTO farmers VALUES (5,'Eswari','9876543214','Trichy',TO_DATE('2025-01-05','YYYY-MM-DD'));
68 INSERT INTO farmers VALUES (6,'Arun','9876543215','Thanjavur',TO_DATE('2025-01-06','YYYY-MM-DD'));
69 INSERT INTO farmers VALUES (7,'Nivetha','9876543216','Namakkal',TO_DATE('2025-01-07','YYYY-MM-DD'));
70 INSERT INTO farmers VALUES (8,'Hari Prasad','9876543217','Karur',TO_DATE('2025-01-08','YYYY-MM-DD'));
```

	FARMER_ID	FARMER_NAME	PHONE	REGION	REGISTRATION_DATE
1	1	Pavithra	9876543210	Coimbatore	1/1/2025, 12:00:00 AM
2	2	Raja	9876543211	Erode	1/2/2025, 12:00:00 AM
3	3	Sankar	9876543212	Salem	1/3/2025, 12:00:00 AM
4	4	Dinesh Kumar	9876543213	Madurai	1/4/2025, 12:00:00 AM
5	5	Eswari	9876543214	Trichy	1/5/2025, 12:00:00 AM
6	6	Arun	9876543215	Thanjavur	1/6/2025, 12:00:00 AM
7	7	Nivetha	9876543216	Namakkal	1/7/2025, 12:00:00 AM

Plots table

INSERT INTO plots VALUES

(1,1,5.50,'North Field','Clay'); INSERT

INTO plots VALUES (2,2,3.75,'East

Field','Loamy'); INSERT INTO plots

VALUES (3,3,4.20,'West Field','Sandy');

177

INSERT INTO plots VALUES (1,1,5.50,'North Field','Clay');

178

INSERT INTO plots VALUES (2,2,3.75,'East Field','Loamy');

179

INSERT INTO plots VALUES (3,3,4.20,'West Field','Sandy');

180

INSERT INTO plots VALUES (4,4,6.10,'South Field','Black Soil');

181

INSERT INTO plots VALUES (5,5,2.80,'River Side','Alluvial');

182

INSERT INTO plots VALUES (6,6,7.25,'Green Valley','Clay');

183

INSERT INTO plots VALUES (7,7,3.60,'Hill Top','Red Soil');

184

INSERT INTO plots VALUES (8,8,5.00,'Lake View','Loamy');

185

INSERT INTO plots VALUES (9,9,4.75,'Sunrise Farm','Sandy');

186

INSERT INTO plots VALUES (10,10,6.50,'Golden Acres','Black Soil');

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.012 seconds

	PLOT_ID	FARMER_ID	AREAS	LOCATION	SOIL_TYPE
1	1	1	5.5	North Field	Clay
2	2	2	3.75	East Field	Loamy
3	3	3	4.2	West Field	Sandy
4	4	4	6.1	South Field	Black Soil
5	5	5	2.8	River Side	Alluvial
6	6	6	7.25	Green Valley	Clay
7	7	7	3.6	Hill Top	Red Soil

Crop table

INSERT INTO crops VALUES

(1,'Rice','Kharif',45.50); INSERT INTO

crops VALUES (2,'Wheat','Rabi',38.20);

INSERT INTO crops VALUES

(3,'Maize','Kharif',50.10);

118	INSERT INTO crops VALUES (1,'Rice','Kharif',45.50);
119	INSERT INTO crops VALUES (2,'Wheat','Rabi',38.20);
120	INSERT INTO crops VALUES (3,'Maize','Kharif',50.10);
121	INSERT INTO crops VALUES (4,'Sugarcane','Annual',800.00);
122	INSERT INTO crops VALUES (5,'Cotton','Kharif',22.50);
123	INSERT INTO crops VALUES (6,'Groundnut','Kharif',18.75);
124	INSERT INTO crops VALUES (7,'Soybean','Kharif',25.60);
125	INSERT INTO crops VALUES (8,'Barley','Rabi',35.40);
126	INSERT INTO crops VALUES (9,'Millet','Kharif',20.80);
127	INSERT INTO crops VALUES (10,'Sunflower','Rabi',16.90);
128	

Query result	Script output	DBMS output	Explain Plan	SQL history
  Download Execution time: 0.013 seconds				
	CROP_ID	NAME	SEASON	AVG_YIELD
1	1	Rice	Kharif	45.5
2	2	Wheat	Rabi	38.2
3	3	Maize	Kharif	50.1
4	4	Sugarcane	Annual	800
5	5	Cotton	Kharif	22.5
6	6	Groundnut	Kharif	18.75
7	7	Soybean	Kharif	25.6
8	8	Barley	Rabi	35.4

Soiltest table

INSERT INTO soiltest VALUES (1, 1, SYSDATE-1, 6.5, 45.2, 18.5, 4.2);

INSERT INTO soiltest VALUES (2, 2, SYSDATE-2, 7.1, 52.3, 20.1, 5.0);

INSERT INTO soiltest VALUES (3, 3, SYSDATE-3, 5.8, 38.4, 15.2, 3.8);

70

71

72

73

74

75

76

77

78

INSERT INTO soiltest VALUES (1, 1, SYSDATE-1, 6.5, 45.2, 18.5, 4.2);

INSERT INTO soiltest VALUES (2, 2, SYSDATE-2, 7.1, 52.3, 20.1, 5.0);

INSERT INTO soiltest VALUES (3, 3, SYSDATE-3, 5.8, 38.4, 15.2, 3.8);

INSERT INTO soiltest VALUES (4, 4, SYSDATE-4, 6.9, 60.5, 25.3, 6.1);

INSERT INTO soiltest VALUES (5, 5, SYSDATE-5, 7.3, 70.2, 30.1, 7.0);

INSERT INTO soiltest VALUES (6, 6, SYSDATE-6, 6.2, 42.6, 19.4, 4.5);

INSERT INTO soiltest VALUES (7, 7, SYSDATE-7, 6.8, 55.1, 22.8, 5.3);

INSERT INTO soiltest VALUES (8, 8, SYSDATE-8, 7.0, 65.0, 28.0, 6.8);

INSERT INTO soiltest VALUES (9, 9, SYSDATE-9, 5.5, 35.7, 14.6, 3.2);

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.02 seconds

	TEST_ID	PLOT_ID	TEST_DATE	PH_LEVEL	NITROGEN	POTASSIUM	ORGANIC_CONTENT
1	1	1	6/15/2026, 4:11:55	6.5	45.2	18.5	4.2
2	2	2	6/14/2026, 4:11:55	7.1	52.3	20.1	5
3	3	3	6/13/2026, 4:11:55	5.8	38.4	15.2	3.8
4	4	4	6/12/2026, 4:11:55	6.9	60.5	25.3	6.1
5	5	5	6/11/2026, 4:11:55	7.3	70.2	30.1	7
6	6	6	6/10/2026, 4:11:55	6.2	42.6	19.4	4.5
7	7	7	6/9/2026, 4:11:55 A	6.8	55.1	22.8	5.3
8	8	8	6/8/2026, 4:11:55 A	7	65	28	6.8

Irrigation_log table

INSERT INTO irrigation_log VALUES (1,1,TO_DATE('01-01-2026','DD-MM-YYYY'),120.50,'Drip');

INSERT INTO irrigation_log VALUES (2,2,TO_DATE('03-01-2026','DD-MM-YYYY'),150.00,'Sprinkler');

INSERT INTO irrigation_log VALUES (3,3,TO_DATE('05-01-2026','DD-MM-YYYY'),100.25,'Flood');

146

147

148

149

150

151

152

153

);

INSERT INTO irrigation_log VALUES (1,1,TO_DATE('01-01-2026','DD-MM-YYYY'),120.50,'Drip');

INSERT INTO irrigation_log VALUES (2,2,TO_DATE('03-01-2026','DD-MM-YYYY'),150.00,'Sprinkler');

INSERT INTO irrigation_log VALUES (3,3,TO_DATE('05-01-2026','DD-MM-YYYY'),100.25,'Flood');

INSERT INTO irrigation_log VALUES (4,4,TO_DATE('07-01-2026','DD-MM-YYYY'),180.75,'Drip');

INSERT INTO irrigation_log VALUES (5,5,TO_DATE('09-01-2026','DD-MM-YYYY'),200.00,'Sprinkler');

INSERT INTO irrigation_log VALUES (6,6,TO_DATE('11-01-2026','DD-MM-YYYY'),130.40,'Flood');

INSERT INTO irrigation_log VALUES (7,7,TO_DATE('13-01-2026','DD-MM-YYYY'),160.60,'Drip');

Query result

Script output

DBMS output

Explain Plan

SQL history

Download

Execution time: 0.011 seconds

	IRRIG_ID	PLOT_ID	IRRIG_DATE	WATER_USED	METHOD
1	1	1	1/1/2026, 12:00:00	120.5	Drip
2	2	2	1/3/2026, 12:00:00	150	Sprinkler
3	3	3	1/5/2026, 12:00:00	100.25	Flood
4	4	4	1/7/2026, 12:00:00	180.75	Drip
5	5	5	1/9/2026, 12:00:00	200	Sprinkler
6	6	6	1/11/2026, 12:00:00	130.4	Flood
7	7	7	1/13/2026, 12:00:00	160.6	Drip

Crop_cycle table

```
INSERT INTO crop_cycle VALUES (1,1,1,TO_DATE('01-01-2026','DD-MM-YYYY'),TO_DATE('01-04-2026','DD-MM-YYYY'),2500,2400,'Harvested',45000);
INSERT INTO crop_cycle VALUES (2,2,2,TO_DATE('03-01-2026','DD-MM-YYYY'),TO_DATE('03-04-2026','DD-MM-YYYY'),2200,2100,'Harvested',42000);
```

```
---
205 INSERT INTO crop_cycle VALUES (1,1,1,TO_DATE('01-01-2026','DD-MM-YYYY'),TO_DATE('01-04-2026','DD-MM-YYYY'),2500,2400,'Harvested',45000);
206 INSERT INTO crop_cycle VALUES (2,2,2,TO_DATE('03-01-2026','DD-MM-YYYY'),TO_DATE('03-04-2026','DD-MM-YYYY'),2200,2100,'Harvested',42000);
207 INSERT INTO crop_cycle VALUES (3,3,3,TO_DATE('05-01-2026','DD-MM-YYYY'),TO_DATE('05-04-2026','DD-MM-YYYY'),2800,2700,'Harvested',48000);
208 INSERT INTO crop_cycle VALUES (4,4,4,TO_DATE('07-01-2026','DD-MM-YYYY'),TO_DATE('07-04-2026','DD-MM-YYYY'),2600,2500,'Harvested',46000);
209 INSERT INTO crop_cycle VALUES (5,5,5,TO_DATE('09-01-2026','DD-MM-YYYY'),TO_DATE('09-04-2026','DD-MM-YYYY'),3000,2900,'Harvested',52000);
---
```

	CYCLE_ID	PLOT_ID	CROP_ID	SOW_DATE	HARVEST_DATE	EXPECTED_YIELD	ACTUAL_YIELD	STATUS	EXPECTED_COST
1	1	1	1	01-JAN-2026	01-APR-2026	2500	2400	Harvested	45000
2	2	2	2	03-JAN-2026	03-APR-2026	2200	2100	Harvested	42000
3	3	3	3	05-JAN-2026	05-APR-2026	2800	2700	Harvested	48000
4	4	4	4	07-JAN-2026	07-APR-2026	2600	2500	Harvested	46000
5	5	5	5	09-JAN-2026	09-APR-2026	3000	2900	Harvested	52000

Fertilizer_logs table

```
INSERT INTO fertilizer_logs VALUES (1,1,TO_DATE('01-01-2026','DD-MM-YYYY'),'Urea',1200,50);
INSERT INTO fertilizer_logs VALUES(2,2,TO_DATE('03-01-2026','DD-MM-YYYY'),'DAP',1500,45);
INSERT INTO fertilizer_logs VALUES (3,3,TO_DATE('05-01-2026','DD-MM-YYYY'),'NPK',1800,40);
```

```
287 INSERT INTO fertilizer_logs VALUES (1,1,TO_DATE('01-01-2026','DD-MM-YYYY'),'Urea',1200,50);
288 INSERT INTO fertilizer_logs VALUES (2,2,TO_DATE('03-01-2026','DD-MM-YYYY'),'DAP',1500,45);
289 INSERT INTO fertilizer_logs VALUES (3,3,TO_DATE('05-01-2026','DD-MM-YYYY'),'NPK',1800,40);
290 INSERT INTO fertilizer_logs VALUES (4,4,TO_DATE('07-01-2026','DD-MM-YYYY'),'Compost',1000,60);
291 INSERT INTO fertilizer_logs VALUES (5,5,TO_DATE('09-01-2026','DD-MM-YYYY'),'Potash',1600,35);
---
```

	FERT_ID	PLOT_ID	FERT_DATE	FERT_TYPE	COST	QUANTITY
1	1	1	01-JAN-2026	Urea	1200	50
2	2	2	03-JAN-2026	DAP	1500	45
3	3	3	05-JAN-2026	NPK	1800	40
4	4	4	07-JAN-2026	Compost	1000	60
5	5	5	09-JAN-2026	Potash	1600	35

Pest_reports table

```
INSERT INTO pest_reports VALUES (1,1,TO_DATE('01-01-2026','DD-MM-YYYY'),'Aphids','Low'); INSERT INTO pest_reports VALUES (2,2,TO_DATE('03-01-2026','DD-MM-YYYY'),'Whiteflies','Medium'); INSERT INTO pest_reports VALUES (3,3,TO_DATE('05-01-2026','DD-MM-YYYY'),'Armyworm','High');
```

```
371 INSERT INTO pest_reports VALUES (1,1,TO_DATE('01-01-2026','DD-MM-YYYY'),'Aphids','Low');
372 INSERT INTO pest_reports VALUES (2,2,TO_DATE('03-01-2026','DD-MM-YYYY'),'Whiteflies','Medium');
373 INSERT INTO pest_reports VALUES (3,3,TO_DATE('05-01-2026','DD-MM-YYYY'),'Armyworm','High');
374 INSERT INTO pest_reports VALUES (4,4,TO_DATE('07-01-2026','DD-MM-YYYY'),'Stem Borer','Severe');
375 INSERT INTO pest_reports VALUES (5,5,TO_DATE('09-01-2026','DD-MM-YYYY'),'Thrips','Low');
376
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.02 seconds

	PEST_ID	PLOT_ID	REPORT_DATE	PEST_TYPE	SEVERITY
1	1	1	01-JAN-2026	Aphids	Low
2	2	2	03-JAN-2026	Whiteflies	Medium
3	3	3	05-JAN-2026	Armyworm	High
4	4	4	07-JAN-2026	Stem Borer	Severe
5	5	5	09-JAN-2026	Thrips	Low

Weather_logs

```
INSERT INTO weather_logs VALUES (1,'North',TO_DATE('01-01-2026','DD-MM-YYYY'),28.5,65,12.5); INSERT INTO weather_logs VALUES (2,'South',TO_DATE('03-01-2026','DD-MM-YYYY'),30.2,68,8.4); INSERT INTO weather_logs VALUES (3,'East',TO_DATE('05-01-2026','DD-MM-YYYY'),27.8,72,15.3);
```

```
441
442 INSERT INTO weather_logs VALUES (1,'North',TO_DATE('01-01-2026','DD-MM-YYYY'),28.5,65,12.5);
443 INSERT INTO weather_logs VALUES (2,'South',TO_DATE('03-01-2026','DD-MM-YYYY'),30.2,68,8.4);
444 INSERT INTO weather_logs VALUES (3,'East',TO_DATE('05-01-2026','DD-MM-YYYY'),27.8,72,15.3);
445 INSERT INTO weather_logs VALUES (4,'West',TO_DATE('07-01-2026','DD-MM-YYYY'),29.4,66,10.2);
446 INSERT INTO weather_logs VALUES (5,'Central',TO_DATE('09-01-2026','DD-MM-YYYY'),31.1,64,5.8);
447
```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.018 seconds

	WLOG_ID	REGION	WEATHER_DATE	TEMPERATURE	HUMIDITY	RAINFALL
1	1	North	01-JAN-2026	28.5	65	12.5
2	2	South	03-JAN-2026	30.2	68	8.4
3	3	East	05-JAN-2026	27.8	72	15.3
4	4	West	07-JAN-2026	29.4	66	10.2
5	5	Central	09-JAN-2026	31.1	64	5.8
6	6	North	11-JAN-2026	28.9	67	13.1

2. Basic Queries

1. Farmers registered in 2024.

SELECT

farmer_id,farmer_name,phone,region,TO_CHAR(registrati
on_date,'DD-MON-YYYY') AS registration_dateFROM
farmersWHERE registration_date >= TO_DATE('01-01-
2024','DD-MM-YYYY') AND registration_date <
TO_DATE('01-01-2025','DD- MM-YYYY');

530

531

532

533

534

SELECT

farmer_id,farmer_name,phone,region,

TO_CHAR(registration_date,'DD-MON-YYYY') AS registration_dateFROM farmers

WHERE registration_date >= TO_DATE('01-01-2024','DD-MM-YYYY')

AND registration_date < TO_DATE('01-01-2025','DD-MM-YYYY');

Query result

Script output

DBMS output

Explain Plan

SQL history





Download

Execution time: 0.009 seconds

	FARMER_ID	FARMER_NAME	PHONE	REGION	REGISTRATION_DA
1	1	Pavithra	9876543210	Coimbatore	01-JAN-2024
2	2	Raja	9876543211	Erode	02-JAN-2024
3	3	Sankar	9876543212	Salem	03-JAN-2024
4	4	Dinesh Kumar	9876543213	Madurai	04-JAN-2024
5	5	Eswari	9876543214	Trichy	05-JAN-2024

2. Plots with soil_type = “clay”.

SELECT * FROM plots WHERE soil_type = 'Clay';

536SELECT *
537FROM plots
538WHERE soil_type = 'Clay';

Query result

Script output

DBMS output

Explain Plan

SQL history

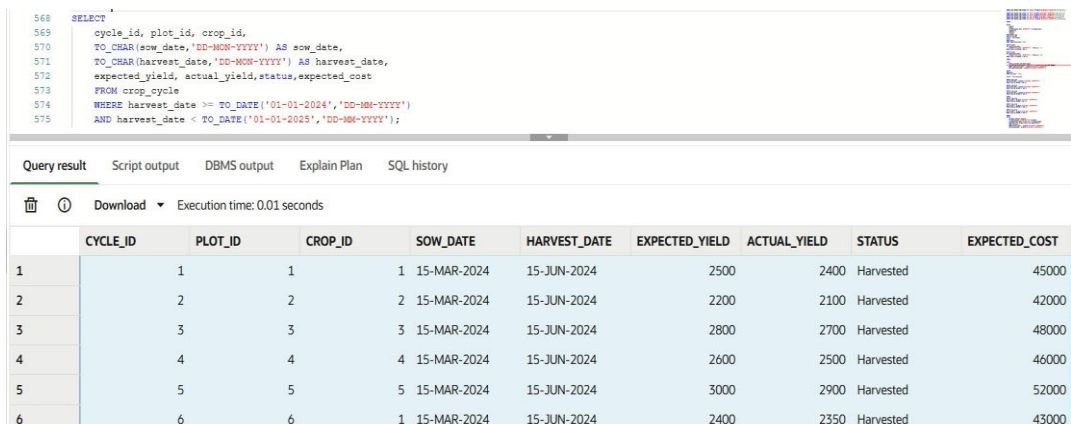
Download

Execution time: 0.008 seconds

	PLOT_ID	FARMER_ID	AREAS	LOCATION	SOIL_TYPE
1	1	1	5.5	North Field	Clay
2	6	6	7.25	Green Valley	Clay
3	11	11	3.25	Palm Grove	Clay
4	17	17	3.85	Forest Edge	Clay
5	22	22	5.15	Royal Acres	Clay
6	26	26	5.25	Green Park	Clay

3.Crops harvested in 2024.

```
SELECT cycle_id, plot_id, crop_id,  
       TO_CHAR(sow_date,'DD-  
MON_YYYY')AS sow_date,  
       TO_CHAR(harvest_date,'DD-MON-YYYY') AS  
harvest_date, expected_yield,  
actual_yield,status,expected_cost FROM crop_cycle  
WHERE harvest_date >= TO_DATE('01-01-2024','DD-  
MM-YYYY') AND harvest_date < TO_DATE('01-01-  
2025','DD-MM-YYYY');
```

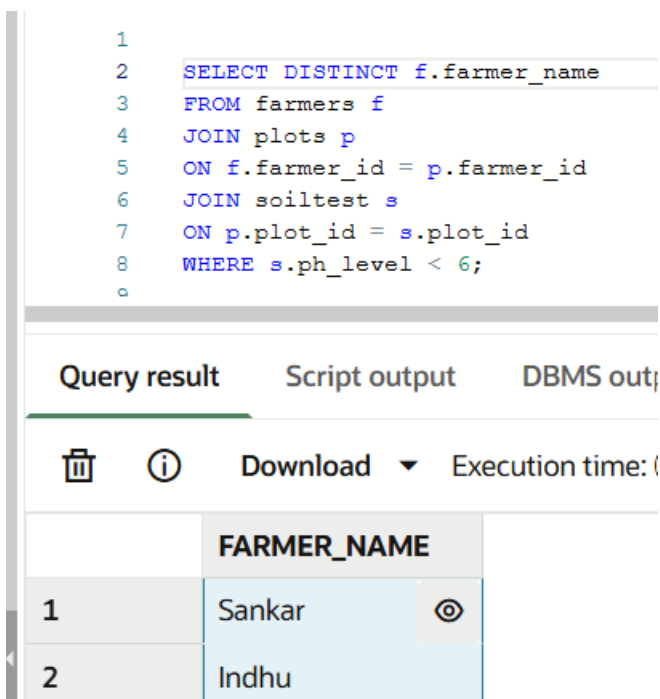


The screenshot shows a database query interface with a SQL editor at the top and a results table below. The SQL query is:
`SELECT cycle_id, plot_id, crop_id, TO_CHAR(sow_date,'DD-MON-YYYY') AS sow_date, TO_CHAR(harvest_date,'DD-MON-YYYY') AS harvest_date, expected_yield, actual_yield, status, expected_cost FROM crop_cycle WHERE harvest_date >= TO_DATE('01-01-2024','DD-MM-YYYY') AND harvest_date < TO_DATE('01-01-2025','DD-MM-YYYY');`
The results table has 10 columns: CYCLE_ID, PLOT_ID, CROP_ID, SOW_DATE, HARVEST_DATE, EXPECTED_YIELD, ACTUAL_YIELD, STATUS, and EXPECTED_COST. It contains 6 rows of data.

	CYCLE_ID	PLOT_ID	CROP_ID	SOW_DATE	HARVEST_DATE	EXPECTED_YIELD	ACTUAL_YIELD	STATUS	EXPECTED_COST
1	1	1	1	15-MAR-2024	15-JUN-2024	2500	2400	Harvested	45000
2	2	2	2	15-MAR-2024	15-JUN-2024	2200	2100	Harvested	42000
3	3	3	3	15-MAR-2024	15-JUN-2024	2800	2700	Harvested	48000
4	4	4	4	15-MAR-2024	15-JUN-2024	2600	2500	Harvested	46000
5	5	5	5	15-MAR-2024	15-JUN-2024	3000	2900	Harvested	52000
6	6	6	6	15-MAR-2024	15-JUN-2024	2400	2350	Harvested	43000

4.Farmers with low ph(<6).

```
SELECT DISTINCT f.farmer_name from farmers  
f JOIN plots p ON f.farmer_id = p.farmer_id  
JOIN soiltest s ON p.plot_id = s.plot_id WHERE  
s.ph_level < 6;
```



The screenshot shows a database query interface with a SQL editor at the top and a results table below. The SQL query is:
`SELECT DISTINCT f.farmer_name FROM farmers f JOIN plots p ON f.farmer_id = p.farmer_id JOIN soiltest s ON p.plot_id = s.plot_id WHERE s.ph_level < 6;`
The results table has 2 columns: FARMER_NAME and an icon column. It contains 2 rows of data.

	FARMER_NAME	
1	Sankar	👁
2	Indhu	

5.Plots with more than 3 pest reports.

SELECT plot_id, COUNT(*) AS total_reports

FROM pest_reports GROUP BY plot_id HAVING COUNT(*) > 3;

Select * from pest_reports;

```
12  SELECT plot_id, COUNT(*) AS total_reports
13  FROM pest_reports
14  GROUP BY plot_id
15  HAVING COUNT(*) > 3;
16  select * from pest_reports;
```

Query result

Script output

DBMS output

E



Download



Execution time: 0.002 sec

	PLOT_ID	TOTAL_REPORTS
1	1	5
2	2	5
3	3	5
4	4	5
5	5	5

Joins & Subqueries

6. Farmers with plot and crop cycle details.

```
SELECT f.farmer_name, p.plot_id, cc.cycle_id,  
       TO_CHAR(cc.sow_date,'DD-MON-YYYY') AS sow_date,  
       TO_CHAR(cc.harvest_date,'DD-MON-YYYY') AS harvest_date,  
       cc.expected_yield,cc.actual_yield,cc.status FROM farmers f JOIN plots p  
       ON f.farmer_id = p.farmer_id JOIN crop_cycle cc ON p.plot_id = cc.plot_id;
```

```
18 SELECT f.farmer_name,  
19       p.plot_id, cc.cycle_id,  
20       TO_CHAR(cc.sow_date,'DD-MON-YYYY') AS sow_date,  
21       TO_CHAR(cc.harvest_date,'DD-MON-YYYY') AS harvest_date,  
22       cc.expected_yield,cc.actual_yield,cc.status  
23 FROM farmers f JOIN plots p  
24 ON f.farmer_id = p.farmer_id  
25 JOIN crop_cycle cc ON p.plot_id = cc.plot_id;
```

	FARMER_NAM	PLOT_ID	CYCLE_ID	SOW_DATE	HARVEST_DATE	EXPECTED_YIELD	ACTUAL_YIELD	STATUS
1	Pavithra	1	1	15-MAR-2024	15-JUN-2024	2500	2400	Harvested
2	Raja	2	2	15-MAR-2024	15-JUN-2024	2200	2100	Harvested
3	Sankar	3	3	15-MAR-2024	15-JUN-2024	2800	2700	Harvested
4	Dinesh Kumar	4	4	15-MAR-2024	15-JUN-2024	2600	2500	Harvested
5	Eswari	5	5	15-MAR-2024	15-JUN-2024	3000	2900	Harvested

7. Crops with yield lower than region average.

```
SELECT cc.cycle_id, f.region, cc.actual_yield  
FROM crop_cycle cc JOIN plots p ON cc.plot_id = p.plot_id JOIN farmers f ON  
p.farmer_id = f.farmer_id  
WHERE cc.actual_yield < (SELECT AVG(cc2.actual_yield) FROM crop_cycle cc2  
JOIN plots p2 ON cc2.plot_id = p2.plot_id JOIN farmers f2 ON  
p2.farmer_id = f2.farmer_idn WHERE f2.region = f.region);
```

```

55 SELECT cc.cycle_id, f.region, cc.actual_yield
56 FROM crop_cycle cc
57 JOIN plots p ON cc.plot_id = p.plot_id
58 JOIN farmers f ON p.farmer_id = f.farmer_id
59 WHERE cc.actual_yield <
60 (
61     SELECT AVG(cc2.actual_yield)
62     FROM crop_cycle cc2
63     JOIN plots p2 ON cc2.plot_id = p2.plot_id
64     JOIN farmers f2 ON p2.farmer_id = f2.farmer_id
65     WHERE f2.region = f.region
66 );

```

Query result	Script output	DBMS output	Explain Plan	SQL
  Download ▾ Execution time: 0.028 seconds				
	CYCLE_ID	REGION	ACTUAL_YIELD	
1	27	Namakkal	1200	
2	29	Tiruppur	1100	
3	30	Dindigul	1450	

8.Plots with fertilizer cost above expected.

```

SELECT p.plot_id, SUM(fl.cost) AS total_fertilizer_cost, cc.expected_cost
FROM plots p JOIN fertilizer_logs fl ON p.plot_id = fl.plot_id
JOIN crop_cycle cc ON p.plot_id = cc.plot_id GROUP BY
p.plot_id, cc.expected_cost HAVING SUM(fl.cost) >
cc.expected_cost;

```

```

68 SELECT p.plot_id,
69 SUM(fl.cost) AS total_fertilizer_cost, cc.expected_cost
70 FROM plots p JOIN fertilizer_logs fl
71 ON p.plot_id = fl.plot_id
72 JOIN crop_cycle cc ON p.plot_id = cc.plot_id
73 GROUP BY p.plot_id, cc.expected_cost
74 HAVING SUM(fl.cost) > cc.expected_cost;|
75

```

Query result	Script output	DBMS output	Explain Plan	SQL
  Download ▾ Execution time: 0.002 seconds				
	PLOT_ID	TOTAL_FERTILIZER	EXPECTED_COST	
1	1	35000	1000	
2	2	42500	1000	
3	3	50000	1000	

9.Regions with heavy rainfall compared to average.

```

SELECT region, rainfall
FROM weather_logs
WHERE rainfall >
( SELECT AVG(rainfall)
FROM weather_logs );

```

```

81
82 SELECT region, rainfall
83 FROM weather_logs
84 WHERE rainfall >
85 (
86     SELECT AVG(rainfall)
87     FROM weather_logs
88 );

```

Query result		Script output	DBMS output
  Download ▼ Execution time: 0.013 se			
	REGION		RAINFALL
1	North		12.5
2	East		15.3
3	North		13.1
4	East		16

10. Farmers not using irrigation.

```

SELECT farmer_name FROM farmers
WHERE farmer_id NOT IN ( SELECT p.farmer_id
FROM plots p JOIN irrigation_logs i ON p.plot_id = i.plot_id );

```

```

92 SELECT farmer_name
93 FROM farmers
94 WHERE farmer_id NOT IN
95 (
96     SELECT p.farmer_id
97     FROM plots p
98     JOIN irrigation_logs i
99     ON p.plot_id = i.plot_id
100 );

```

Query result		Script output	DBMS c
  Download ▼ Execution tim			
	FARMER_NAME		
1	Karthik		
2	Lakshmi		
3	Manoj		

Reports

11.Top 10 high-yield farmers.

```
SELECT farmer_name, total_yield
FROM ( SELECT f.farmer_name, SUM(cc.actual_yield) AS total_yield
      FROM farmers f JOIN plots p ON f.farmer_id = p.farmer_id JOIN
      crop_cycle cc
      ON p.plot_id = cc.plot_id GROUP BY f.farmer_name ORDER BY
      total_yield DESC) WHERE ROWNUM <= 10;
```

102	SELECT farmer_name, total_yield
103	FROM (
104	SELECT f.farmer_name, SUM(cc.actual_yield)
105	AS total_yield FROM farmers f
106	JOIN plots p ON f.farmer_id = p.farmer_id
107	JOIN crop_cycle cc ON p.plot_id = cc.plot_id
108	GROUP BY f.farmer_name ORDER BY total_yield DESC
109	)
110	WHERE ROWNUM <= 10;

Query result	Script output	DBMS output	Explain Plan
  Download ▾ Execution time: 0.016 seconds			
	FARMER_NAME		TOTAL_YIELD
1	Sankar		9330
2	Eswari		9130
3	Raja		8600
4	Pavithra		8510
5	Dinesh Kumar		8240

12.Region-wise fertilizer cost summary.

```
SELECT f.region, SUM(fl.cost) AS total_fertilizer_cost
FROM farmers f JOIN plots p ON f.farmer_id = p.farmer_id
JOIN fertilizer_logs fl ON p.plot_id = fl.plot_id GROUP BY
f.region ORDER BY f.region;
```

112	SELECT f.region,
113	SUM(fl.cost) AS total_fertilizer_cost
114	FROM farmers f JOIN plots p
115	ON f.farmer_id = p.farmer_id
116	JOIN fertilizer_logs fl
117	ON p.plot_id = fl.plot_id
118	GROUP BY f.region ORDER BY f.region;

Query result	Script output	DBMS output	Explain Pla
  Download ▾ Execution time: 0.015 seconds			
	REGION		TOTAL_FERTILIZER_COST
1	Coimbatore		7000
2	Dindigul		9250
3	Erode		8500
4	Karur		10250

13. Plot with highest irrigation usage.

```
SELECT plot_id, SUM(water_used) AS total_water_used
FROM irrigation_logs GROUP BY plot_id
ORDER BY total_water_used DESC
FETCH FIRST 1 ROW ONLY;
```

```
120 SELECT plot_id, SUM(water_used) AS total_water_used
121 FROM irrigation_logs
122 GROUP BY plot_id
123 ORDER BY total_water_used DESC
124 FETCH FIRST 1 ROW ONLY;
```

Query result	Script output	DBMS output	Explain Plan
  Download ▾ Execution time: 0.01 seconds			
	PLOT_ID	TOTAL_WATER_US	
1	5	2221	

14. Pest severity distribution by region.

```
SELECT f.region, pr.severity, COUNT(*) AS total_reports
FROM farmers f JOIN plots p ON f.farmer_id = p.farmer_id
JOIN pest_reports pr ON p.plot_id = pr.plot_id
GROUP BY f.region, pr.severity
ORDER BY f.region, pr.severity;
```

```
126 SELECT f.region, pr.severity, COUNT(*) AS total_reports
127 FROM farmers f JOIN plots p ON f.farmer_id = p.farmer_id
128 JOIN pest_reports pr ON p.plot_id = pr.plot_id
129 GROUP BY f.region, pr.severity
130 ORDER BY f.region, pr.severity;
```

Query result	Script output	DBMS output	Explain Plan	SQL
  Download ▾ Execution time: 0.019 seconds				
	REGION	SEVERITY	TOTAL_REPORTS	
1	Coimbatore	High	2	
2	Coimbatore	Low	3	
3	Dindigul	Medium	3	
4	Dindigul	Severe	2	

15.Crops with repeated low actual yield.

```
SELECT c.name, COUNT(*) AS  
low_yield_count FROM crops c JOIN  
crop_cycle cc  
ON c.crop_id = cc.crop_id WHERE  
cc.actual_yield < 1000 GROUP BY c.name  
HAVING COUNT(*) > 1;
```

```
131  
132 SELECT c.name,  
133 COUNT(*) AS low_yield_count  
134 FROM crops c  
135 JOIN crop_cycle cc  
136 ON c.crop_id = cc.crop_id  
137 WHERE cc.actual_yield < 1000  
138 GROUP BY c.name  
139 HAVING COUNT(*) > 1;
```

Query result			Script output	DBMS output
		Download	▼	Execution time: 0.002 se
	NAME		LOW_YIELD_COUN	
1	Rice		3	
2	Wheat		3	
3	Maize		3	
4	Sugarcane		4	

Advanced

16. Monthly rainfall report.

```
SELECT TO_CHAR(weather_date, 'MON  
YYYY') AS month, SUM(rainfall) AS  
total_rainfall FROM weather_logs  
GROUP BY TO_CHAR(weather_date, 'MON-YYYY')
```

```
143 SELECT TO_CHAR(weather_date, 'MON-YYYY') AS month,  
144 SUM(rainfall) AS total_rainfall  
145 FROM weather_logs  
146 GROUP BY TO_CHAR(weather_date, 'MON-YYYY')  
147 ORDER BY MIN(weather_date);  
148
```

Query result				Script output	DBMS output	Explain Plan
		Download	▼	Execution time: 0.011 seconds		
	MONTH		TOTAL_RAINFALL			
1	JAN-2026		167.7			
2	FEB-2026		145			
3	MAR-2026		158.3			
4	APR-2026		51.7			

17. Farmers earning above average yield per area.

```
SELECT f.farmer_name,  
SUM(cc.actual_yield)/SUM(p.areas) AS  
yield_per_area FROM farmers f JOIN  
plots p ON f.farmer_id = p.farmer_id  
JOIN crop_cycle cc ON p.plot_id =  
cc.plot_id GROUP BY f.farmer name;
```

```
171 SELECT f.farmer_name,  
172 SUM(cc.actual_yield)/SUM(p.areas) AS yield_per_area  
173 FROM farmers f JOIN plots p  
174 ON f.farmer_id = p.farmer_id JOIN crop_cycle cc  
175 ON p.plot_id = cc.plot_id GROUP BY f.farmer_name;
```

Query result	Script output	DBMS output	Explain Plan
  Download ▾ Execution time: 0.01 seconds			
	FARMER_NAME	YIELD_PER_AREA	
1	Pavithra	309.45454545454544	
2	Raja	458.66666666666667	
3	Sankar	444.2857142857143	
4	Dinesh Kumar	270.1639344262295	

18. Plots whose yield increased over seasons.

```
SELECT plot_id,cycle_id, actual_yield,prev_yield FROM  
( SELECT plot_id, cycle_id,  
actual_yield,LAG(actual_yield) OVER ( PARTITION  
BY plot_id ORDER BY cycle_id) AS prev_yield  
FROM crop_cycle) WHERE actual_yield > prev_yield;
```



```

204 SELECT plot_id,cycle_id,
205 actual_yield,prev_yield
206 FROM (
207 SELECT plot_id, cycle_id,
208 actual_yield,LAG(actual_yield) OVER (
209 PARTITION BY plot_id ORDER BY cycle_id)
210 AS prev_yield FROM crop_cycle)
211 WHERE actual_yield > prev_yield;
212

```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.002 seconds

	PLOT_ID	CYCLE_ID	ACTUAL_YIELD	PREV_YIELD
1	1	21	2480	2380
2	2	12	2550	2100
3	2	22	2650	2550
4	3	23	2800	2680

19. Farmers using only organic fertilizers.

```

SELECT DISTINCT f.farmer_name, fl.fert_type
FROM farmers f JOIN plots p ON
f.farmer_id = p.farmer_id JOIN
fertilizer_logs fl ON p.plot_id = fl.plot_id
WHERE fl.fert_type = 'Organic' AND NOT EXISTS
( SELECT 1 FROM plots p2 JOIN fertilizer_logs fl2
ON p2.plot_id = fl2.plot_id WHERE
p2.farmer_id = f.farmer_id AND fl2.fert_type <>
'Organic' );

```

```

348 SELECT DISTINCT f.farmer_name, fl.fert_type
349 FROM farmers f JOIN plots p
350 ON f.farmer_id = p.farmer_id
351 JOIN fertilizer_logs fl
352 ON p.plot_id = fl.plot_id
353 WHERE fl.fert_type = 'Organic'
354 AND NOT EXISTS
355 (
356 SELECT 1 FROM plots p2
357 JOIN fertilizer_logs fl2
358 ON p2.plot_id = fl2.plot_id
359 WHERE p2.farmer_id = f.farmer_id
360 AND fl2.fert_type <> 'Organic'
361 );
362

```

Query result Script output DBMS output Explai

Download Execution time: 0.014 seconds

	FARMER_NAME	FERT_TYPE
1	Pavithra	Organic
2	Raja	Organic
3	Sankar	Organic

20. View: Farmer, Total Yield, Avg Soil pH, Fertilizer Cost.

```
CREATE VIEW farmer_summary AS SELECT f.farmer_name,  
SUM(cc.actual_yield) AS total_yield, AVG(s.ph_level) AS  
avg_soil_ph, SUM(fl.cost) AS fertilizer_cost FROM farmers f  
JOIN plots p ON f.farmer_id = p.farmer_id JOIN crop_cycle cc  
ON p.plot_id = cc.plot_id JOIN soiltest s ON p.plot_id = s.plot_id  
JOIN fertilizer_logs fl ON p.plot_id = fl.plot_id GROUP BY  
f.farmer_name; SELECT * FROM farmer_summary;
```

```
376 CREATE VIEW farmer_summary AS  
377 SELECT f.farmer_name, SUM(cc.actual_yield) AS total_yield,  
378 AVG(s.ph_level) AS avg_soil_ph, SUM(fl.cost) AS fertilizer_cost  
379 FROM farmers f JOIN plots p ON f.farmer_id = p.farmer_id  
380 JOIN crop_cycle cc ON p.plot_id = cc.plot_id JOIN soiltest s  
381 ON p.plot_id = s.plot_id JOIN fertilizer_logs fl  
382 ON p.plot_id = fl.plot_id GROUP BY f.farmer_name;  
383  
384 SELECT * FROM farmer_summary;
```

	FARMER_NAME	TOTAL_YIELD	AVG_SOIL_PH	FERTILIZER_COST
1	Pavithra	212750	6.56	175000
2	Raja	215000	7.12	212500
3	Sankar	233250	5.82	250000
4	Dinesh Kumar	206000	6.76	150000
5	Eswari	228250	7.38	225000

3. Optimization

Indexes:

- plot_id • crop_id • region • date fields

```
1 CREATE INDEX idx_plot_id ON plots(plot_id);  
2  
3 CREATE INDEX idx_crop_id ON crop_cycle(crop_id);  
4  
5 CREATE INDEX idx_region ON weather_logs(region);  
6  
7 CREATE INDEX idx_weather_date ON weather_logs(weather_date);  
8  
9 CREATE INDEX idx_fert_date ON fertilizer_logs(fert_date);  
10  
11 CREATE INDEX idx_report_date ON pest_reports(report_date);  
12
```

Query result	Script output	DBMS output	Explain Plan	SQL history
Index IDX_WEATHER_DATE created. Elapsed: 00:00:00.015				
SQL> CREATE INDEX idx_fert_date ON fertilizer_logs(fert_date)				
Index IDX_FERT_DATE created. Elapsed: 00:00:00.015				

Rewrite two queries (e.g., fertilizer cost report and yield comparison) using index

SELECT p.plot_id, SUM(fl.cost) AS

total_fertilizer_cost FROM plots p JOIN

fertilizer_logs fl ON p.plot_id = fl.plot_id

GROUP BY p.plot_id;

```
34 SELECT p.plot_id,
35 SUM(fl.cost) AS total_fertilizer_cost
36 FROM plots p JOIN fertilizer_logs fl
37 ON p.plot_id = fl.plot_id
38 GROUP BY p.plot_id;
```

Query result	Script output	DBMS output
  Download ▾ Execution time: 0.012 se		
	PLOT_ID	TOTAL_FERTILIZER
1	1	7000
2	2	8500
3	3	10000
4	4	6000

SELECT c.name,

AVG(cc.actual_yield) AS avg_yield

FROM crops c JOIN crop_cycle cc

ON c.crop_id= cc.crop_id GROUP BY

c.name;

```
40 SELECT c.name,
41 AVG(cc.actual_yield) AS avg_yield
42 FROM crops c
43 JOIN crop_cycle cc
44 ON c.crop_id = cc.crop_id
45 GROUP BY c.name;
```

Query result	Script output	DBMS output
  Download ▾ Execution time: 0.011 se		
	NAME	AVG_YIELD
1	Rice	1586
2	Wheat	1488
3	Maize	1608
4	Sugarcane	1504

4. Transactions & Recovery

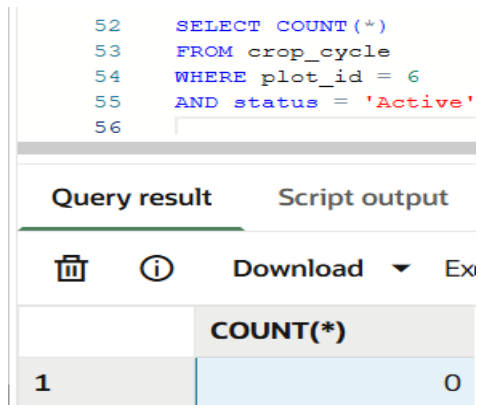
Crop Cycle Creation Transaction

1. Validate no active crop on the plot

SELECT

COUNT(*) FROM crop_cycle WHERE

plot_id = 6 AND status = 'Active';



The screenshot shows a SQL query editor with the following code:

```
52 SELECT COUNT(*)
53 FROM crop_cycle
54 WHERE plot_id = 6
55 AND status = 'Active'
56
```

Below the editor, there are two tabs: "Query result" (selected) and "Script output". Under the "Query result" tab, there is a table with one row and one column:

COUNT(*)
0

2. Insert cycle record

INSERT INTO crop_cycle

(cycle_id, plot_id, crop_id, sow_date, harvest_date,
expected_yield, actual_yield, status, expected_cost)

VALUES (53,6,2, TO_DATE('10-07-2026','DD-

MM-YYYY'),

TO_DATE('10-10-2026','DD-MM-YYYY'),3000,2850, 'Active',45000);

```

67 | INSERT INTO crop_cycle
68 | (cycle_id, plot_id, crop_id, sow_date, harvest_date,
69 | expected_yield, actual_yield, status, expected_cost)
70 | VALUES
71 | (53,6,2,
72 | TO_DATE('10-07-2026','DD-MM-YYYY'),
73 | TO_DATE('10-10-2026','DD-MM-YYYY'),3000,2850,
74 | 'Active',45000
75 | );

```

```

SQL> INSERT INTO crop_cycle
      (cycle_id, plot_id, crop_id, sow_date, harvest_date,
      expected_yield, actual_yield, status, expected_cost)
      VALUES...
Show more...

```

1 row inserted.

Elapsed: 00:00:00.011

3. Insert expected yield

INSERT INTO crop_cycle

(cycle_id, plot_id, crop_id, sow_date, harvest_date,

expected_yield, actual_yield, status,

expected_cost) VALUES (53,6,2, TO_DATE('10-

07-2026','DD-MM-YYYY'),

TO_DATE('10-10-2026','DD-MM-YYYY'),3000,2850, 'Active',45000);

```

67 | INSERT INTO crop_cycle
68 | (cycle_id, plot_id, crop_id, sow_date, harvest_date,
69 | expected_yield, actual_yield, status, expected_cost)
70 | VALUES
71 | (53,6,2,
72 | TO_DATE('10-07-2026','DD-MM-YYYY'),
73 | TO_DATE('10-10-2026','DD-MM-YYYY'),3000,2850,
74 | 'Active',45000
75 | );

```

```

SQL> INSERT INTO crop_cycle
      (cycle_id, plot_id, crop_id, sow_date, harvest_date,
      expected_yield, actual_yield, status, expected_cost)
      VALUES...
Show more...

```

1 row inserted.

Elapsed: 00:00:00.011

4. Commit

Commit;

```
79 | commit;
```

Query result

Script output



SQL> commit

Commit complete.

Elapsed: 00:00:00.001

Demonstrations

- **SAVEPOINT before yield entry**

START TRANSACTION;

UPDATE crop_cycle SET

actual_yield = 4500 WHERE

cycle_id = 1; SAVEPOINT

yield_entry; UPDATE crop_cycle

SET actual_yield = 4700 WHERE

cycle_id = 1; ROLLBACK TO

yield_entry; COMMIT;

```
11 START TRANSACTION;
12 UPDATE crop_cycle
13 SET actual_yield = 4500
14 WHERE cycle_id = 1;
15 SAVEPOINT yield_entry;
16 UPDATE crop_cycle
17 SET actual_yield = 4700
18 WHERE cycle_id = 1;
19 ROLLBACK TO yield_entry;
20 COMMIT;
```

Output			
Action Output			
#	Time	Action	
✓ 29	11:47:54	CREATE TABLE weather_logs (wlog_id INT PRIMARY KEY, region VARCHAR(30), weat	
✓ 30	11:48:15	INSERT INTO weather_logs (wlog_id, region, weather_date, temperature, humidity, rainfall) VALU	
✓ 31	11:50:45	START TRANSACTION	
✓ 32	11:50:45	UPDATE crop_cycle SET actual_yield = 4500 WHERE cycle_id = 1	
✓ 33	11:50:45	SAVEPOINT yield_entry	
✓ 34	11:50:45	UPDATE crop_cycle SET actual_yield = 4700 WHERE cycle_id = 1	
✓ 35	11:50:45	ROLLBACK TO yield_entry	
✓ 36	11:50:45	COMMIT	

- **ROLLBACK if duplicate cycle**

START TRANSACTION;INSERT INTO crop_cycle

(cycle_id, plot_id, crop_id, sow_date, harvest_date,

expected_yield, actual_yield, status,

expected_cost) VALUES (21,1,1,'2025-06-

01','2025-09-30', 4500,0,'Planned',50000);

```

22 • START TRANSACTION;
23 • INSERT INTO crop_cycle
24 (cycle_id, plot_id, crop_id, sow_date, harvest_date,
25 expected_yield, actual_yield, status, expected_cost)
26 VALUES
27 (21, 1, 1, '2025-06-01', '2025-09-30',
28 4500, 0, 'Planned', 50000);
29 • ROLLBACK;

```

Output			
Action Output			
#	Time	Action	
✓ 32	11:50:45	UPDATE crop_cycle SET actual_yield = 4500 WHERE cycle_id = 1	
✓ 33	11:50:45	SAVEPOINT yield_entry	
✓ 34	11:50:45	UPDATE crop_cycle SET actual_yield = 4700 WHERE cycle_id = 1	
✓ 35	11:50:45	ROLLBACK TO yield_entry	
✓ 36	11:50:45	COMMIT	
✓ 37	12:03:38	START TRANSACTION	
✓ 38	12:03:38	INSERT INTO crop_cycle (cycle_id, plot_id, crop_id, sow_date, harvest_date, expected_yield, actual_yield, st...	
✓ 39	12:03:38	ROLLBACK	

- **Concurrency conflict: two officers assigning cycle to same plot**

START TRANSACTION;

SELECT * FROM plot WHERE plot_id = 5 FOR UPDATE;

INSERT INTO crop_cycle (cycle_id, plot_id, crop_id,

sow_date, harvest_date, expected_yield, actual_yield, status,

expected_cost) VALUES

(22, 5, 4, '2025-06-02', '2025-10-02', 5500, 0, 'Planned', 65000);

```

31 • START TRANSACTION;
32 • SELECT *
33 FROM plot
34 WHERE plot_id = 5
35 FOR UPDATE;
36 • INSERT INTO crop_cycle
37 (cycle_id, plot_id, crop_id, sow_date, harvest_date,
38 expected_yield, actual_yield, status, expected_cost)
39 VALUES
40 (21, 5, 3, '2025-06-01', '2025-10-01',
41 5000, 0, 'Planned', 60000);
42 • commit;

```

Output			
Action Output			
#	Time	Action	Message
✓ 38	12:03:38	INSERT INTO crop_cycle (cycle_id, plot_id, crop_id, sow_date, harvest_date, expected_yield, actual_yield, st...	1 row(s) affected
✓ 39	12:03:38	ROLLBACK	0 row(s) affected
✓ 40	12:11:38	START TRANSACTION	0 row(s) affected
✓ 41	12:11:38	SELECT * FROM plot WHERE plot_id = 5 LIMIT 0, 50000 FOR UPDATE	1 row(s) returned
✓ 42	12:11:38	INSERT INTO crop_cycle (cycle_id, plot_id, crop_id, sow_date, harvest_date, expected_yield, actual_yield, st...	1 row(s) affected
✓ 43	12:13:39	SELECT * FROM plot WHERE plot_id = 5 LIMIT 0, 50000 FOR UPDATE	1 row(s) returned
✗ 44	12:13:40	INSERT INTO crop_cycle (cycle_id, plot_id, crop_id, sow_date, harvest_date, expected_yield, actual_yield, st...	Error Code: 1062. Duplicate entry '21' for
✓ 45	12:14:05	commit	0 row(s) affected


```

2 • START TRANSACTION;
3 • SELECT * FROM plot
4 • WHERE plot_id = 5 FOR UPDATE;
5 • INSERT INTO crop_cycle
6 • (cycle_id, plot_id, crop_id, sow_date, harvest_date,
7 • expected_yield, actual_yield, status, expected_cost)
8 • VALUES
9 • (22, 5, 4, '2025-06-02', '2025-10-02',
10 • 5500, 0, 'Planned', 65000);

```

Result Grid					
	plot_id	farmer_id	areas	location	soil_type
▶	5	5	2.75	Truppur Central	Red Soil
•	MULCH	MULCH	MULCH	MULCH	MULCH

Output			
Action Output			
#	Time	Action	Message
✓	1 12:08:05	USE agri	0 row(s) affected
✓	2 12:12:22	START TRANSACTION	0 row(s) affected
✗	3 12:12:22	SELECT * FROM plot WHERE plot_id = 5 LIMIT 0, 50000 FOR UPDATE	Error Code: 2013. Los
✗	4 12:12:52	INSERT INTO crop_cycle (cycle_id, plot_id, crop_id, sow_date, harvest_date, expected_yield, actual_yield, sta...	Error Code: 2013. Los
✓	5 12:13:48	START TRANSACTION	0 row(s) affected
✓	6 12:13:48	SELECT * FROM plot WHERE plot_id = 5 LIMIT 0, 50000 FOR UPDATE	1 row(s) returned
✓	7 12:14:05	INSERT INTO crop_cycle (cycle_id, plot_id, crop_id, sow_date, harvest_date, expected_yield, actual_yield, sta...	1 row(s) affected

• Deadlock: irrigation update + soil test update

START TRANSACTION;

UPDATE irrigation_log SET

water_used = 2500 WHERE

irrig_id = 1;

UPDATE soiltest SET nitrogen = 145 WHERE test_id = 1;

```

44 • START TRANSACTION;
45 • UPDATE irrigation_log
46 • SET water_used = 2500
47 • WHERE irrig_id = 1;
48 • UPDATE soiltest
49 • SET nitrogen = 145
50 • WHERE test_id = 1;

```

Output			
Action Output			
#	Time	Action	Message
✓	41 12:11:38	SELECT * FROM plot WHERE plot_id = 5 LIMIT 0, 50000 FOR UPDATE	1 row(s) returned
✓	42 12:11:38	INSERT INTO crop_cycle (cycle_id, plot_id, crop_id, sow_date, harvest_date, expected_yield, actual_yield, st...	1 row(s) affected
✓	43 12:13:39	SELECT * FROM plot WHERE plot_id = 5 LIMIT 0, 50000 FOR UPDATE	1 row(s) returned
✗	44 12:13:40	INSERT INTO crop_cycle (cycle_id, plot_id, crop_id, sow_date, harvest_date, expected_yield, actual_yield, st...	Error Code: 1062. Duplicate entry '21' for key 'crop_cycle.PRIMARY'
✓	45 12:14:05	commit	0 row(s) affected
✓	46 12:25:15	START TRANSACTION	0 row(s) affected
✓	47 12:25:15	UPDATE irrigation_log SET water_used = 2500 WHERE irrig_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓	48 12:25:26	UPDATE soiltest SET nitrogen = 145 WHERE test_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

```

START TRANSACTION;

UPDATE soiltest SET nitrogen

= 140 WHERE test_id = 1;

UPDATE

irrigation_log SET

water_used = 2600

WHERE irrig_id =

1;

```

```

12 • START TRANSACTION;
13 • UPDATE soiltest
14   SET nitrogen = 140
15   WHERE test_id = 1;
16
17 • UPDATE irrigation_log
18   SET water_used = 2600
19   WHERE irrig_id = 1;

```

Output

#	Time	Action	Message
11	12:24:45	INSERT INTO crop_cycle (cycle_id, plot_id, crop_id, sow_date, harvest_date, expected_yield, actual_yield, s...	Error Code: 1062. Duplicate entry '22' for key 'crop_cycle.PRIMARY'
12	12:24:45	START TRANSACTION	0 row(s) affected
13	12:24:45	UPDATE soiltest SET nitrogen = 140 WHERE test_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
14	12:24:45	UPDATE irrigation_log SET water_used = 2600 WHERE irrig_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
15	12:25:26	START TRANSACTION	0 row(s) affected
16	12:25:26	UPDATE soiltest SET nitrogen = 140 WHERE test_id = 1	0 row(s) affected Rows matched: 1 Changed: 0 Warnings: 0
17	12:25:26	UPDATE irrigation_log SET water_used = 2600 WHERE irrig_id = 1	Error Code: 1213. Deadlock found when trying to get lock; try restarting transaction

• Log-based recovery

RECOVERY RESOLUTION

MySQL automatically:

- Detects incomplete transactions
- Performs UNDO for uncommitted transactions
- Performs REDO for committed transactions
- Restores database consistency

RESULT AFTER RECOVERY

- Committed updates are preserved
- Incomplete updates are rolled back
- No data loss for committed transactions
- Database remains in a consistent state
- No partial updates remain in the system

5. Security

(a) Admin Role

The Admin has complete control over the Agri Management System and is responsible for database administration and security management.

Permissions assigned:

- Full access to all tables
- Create, modify, and delete users
- Grant and revoke privileges
- Monitor database

activities CREATE ROLE

'admin_role';

GRANT ALL PRIVILEGES ON Agri.* TO 'admin_role';

```
20 CREATE ROLE 'admin_role';
21 GRANT ALL PRIVILEGES ON Agri.* TO 'admin_role';
```

🗲 Action output

#	Time	Action	Message
✓ 18	12:31:02	CREATE ROLE 'admin_role'	0 row(s) affected
✓ 19	12:31:02	GRANT ALL PRIVILEGES ON Agri.* TO 'admin_r...	0 row(s) affected

(b) Agronomist Role

The Agronomist is responsible for analyzing soil conditions, recommending crops, and monitoring pest-related activities.

Permissions assigned:

- View farmer and plot details
- View and update soil test reports
- View crop cycle information
- View pest reports

CREATE ROLE

'agronomist_role';

GRANT SELECT ON Agri.farmers TO

'agronomist_role'; GRANT SELECT ON

Agri.plot TO 'agronomist_role';

GRANT SELECT, UPDATE ON Agri.soiltest TO

'agronomist_role'; GRANT SELECT ON

Agri.crop_cycle TO 'agronomist_role'; GRANT

SELECT ON Agri.pest_reports TO

'agronomist_role';

```

22 CREATE ROLE 'agronomist_role';
23 GRANT SELECT ON Agri.farmers TO 'agronomist_role';
24 GRANT SELECT ON Agri.plot TO 'agronomist_role';
25 GRANT SELECT, UPDATE ON Agri.soiltest TO 'agronomist_role';
26 GRANT SELECT ON Agri.crop_cycle TO 'agronomist_role';
27 GRANT SELECT ON Agri.pest_reports TO 'agronomist_role';

```

⌘ Action output

#	Time	Action	Message
20	12:42:10	CREATE ROLE 'agronomist_role'	0 row(s) affected
21	12:42:10	GRANT SELECT ON Agri.farmers TO 'agronomis...	0 row(s) affected
22	12:42:10	GRANT SELECT ON Agri.plot TO 'agronomist_r...	0 row(s) affected
23	12:42:10	GRANT SELECT, UPDATE ON Agri.soiltest TO '...	0 row(s) affected
24	12:42:10	GRANT SELECT ON Agri.crop_cycle TO 'agrono...	0 row(s) affected
25	12:42:10	GRANT SELECT ON Agri.pest_reports TO 'agro...	0 row(s) affected

(c) Farmer Role

The Farmer can access only their farming-related information. Permissions assigned:

- View plot information
- View crop cycle details
- View soil test reports
- View weather

information CREATE ROLE

'farmer_role';

GRANT SELECT ON Agri.plot TO

'farmer_role'; GRANT SELECT ON

Agri.crop_cycle TO 'farmer_role'; GRANT

SELECT ON Agri.soiltest TO

'farmer_role';

GRANT SELECT ON Agri.weather_logs TO

'farmer_role';

Creation of Users

CREATE USER 'admin1'@'localhost' IDENTIFIED BY

'admin123'; CREATE USER 'agronomist1'@'localhost'

IDENTIFIED BY 'agro123'; CREATE USER

'farmer1'@'localhost' IDENTIFIED BY 'farmer123';

CREATE USER 'weather1'@'localhost' IDENTIFIED BY

'weather123';

```

28 CREATE ROLE 'farmer_role';
29 GRANT SELECT ON Agri.plot TO 'farmer_role';
30 GRANT SELECT ON Agri.crop_cycle TO 'farmer_role';
31 GRANT SELECT ON Agri.soiltest TO 'farmer_role';
32 GRANT SELECT ON Agri.weather_logs TO 'farmer_role';
33
34 -- Creation of users
35 CREATE USER 'admin1'@'localhost' IDENTIFIED BY 'admin123';
36 CREATE USER 'agronomist1'@'localhost' IDENTIFIED BY 'agro123';
37 CREATE USER 'farmer1'@'localhost' IDENTIFIED BY 'farmer123';
38 CREATE USER 'weather1'@'localhost' IDENTIFIED BY 'weather123';

```

⌘ Action output

#	Time	Action	Message
26	12:50:33	CREATE ROLE 'farmer_role'	0 row(s) affected
27	12:50:33	GRANT SELECT ON Agri.plot TO 'farmer_role'	0 row(s) affected
28	12:50:33	GRANT SELECT ON Agri.crop_cycle TO 'farmer...	0 row(s) affected
29	12:50:33	GRANT SELECT ON Agri.soiltest TO 'farmer_r...	0 row(s) affected
30	12:50:33	GRANT SELECT ON Agri.weather_logs TO 'farm...	0 row(s) affected
31	12:50:33	CREATE USER 'admin1'@'localhost' IDENTIFIE...	0 row(s) affected
32	12:50:33	CREATE USER 'agronomist1'@'localhost' IDEN...	0 row(s) affected
33	12:50:33	CREATE USER 'farmer1'@'localhost' IDENTIFI...	0 row(s) affected
34	12:50:33	CREATE USER 'weather1'@'localhost' IDENTIF...	0 row(s) affected

Assignment of Roles to Users

```

GRANT 'admin_role' TO 'admin1'@'localhost';
GRANT 'agronomist_role' TO
'agronomist1'@'localhost'; GRANT
'farmer_role' TO 'farmer1'@'localhost';
GRANT 'weather_officer_role' TO 'weather1'@'localhost';

```

```

39 GRANT 'admin_role' TO 'admin1'@'localhost';
40 GRANT 'agronomist_role' TO 'agronomist1'@'localhost';
41 GRANT 'farmer_role' TO 'farmer1'@'localhost';
42 GRANT 'weather_officer_role' TO 'weather1'@'localhost';

```

⌘ Action output

#	Time	Action	Message
35	13:02:47	GRANT 'admin_role' TO 'admin1'@...	0 row(s) affected
36	13:02:47	GRANT 'agronomist_role' TO 'agr...	0 row(s) affected
37	13:02:47	GRANT 'farmer_role' TO 'farmer1...	0 row(s) affected
38	13:02:47	GRANT 'weather_officer_role' TO...	Error Code: 3523. Unknown authorization ID 'weather_officer_role'@'%'

Activation of Roles

```

SET DEFAULT ROLE 'admin_role' TO 'admin1'@'localhost';
SET DEFAULT ROLE 'agronomist_role' TO
'agronomist1'@'localhost'; SET DEFAULT ROLE
'farmer_role' TO 'farmer1'@'localhost';
SET DEFAULT ROLE 'weather_officer_role' TO 'weather1'@'localhost';

```

```

43 SET DEFAULT ROLE 'admin_role' TO 'admin1'@'localhost';
44 SET DEFAULT ROLE 'agronomist_role' TO 'agronomist1'@'localhost';
45 SET DEFAULT ROLE 'farmer_role' TO 'farmer1'@'localhost';
46 SET DEFAULT ROLE 'weather_officer_role' TO 'weather1'@'localhost';

```

⊞ Action output

#	Time	Action	Message
✓ 39	13:09:18	SET DEFAULT ROLE 'admin_role' TO '...	0 row(s) affected
✓ 40	13:09:18	SET DEFAULT ROLE 'agronomist_role'...	0 row(s) affected
✓ 41	13:09:18	SET DEFAULT ROLE 'farmer_role' TO ...	0 row(s) affected
⊗ 42	13:09:18	SET DEFAULT ROLE 'weather_officer_...	Error Code: 3523. Unknown authorization ID 'weather_officer_role'@'%'

Revoking Permissions

If a role should lose certain privileges, the REVOKE command is used.

For example, removing soil test update access from the Agronomist role:

REVOKE UPDATE ON Agri.soiltest FROM 'agronomist_role';

This ensures that the Agronomist can only view soil test information

and cannot modify it. SQL Injection Demonstration

```

47 REVOKE UPDATE ON Agri.soiltest FROM 'agronomist_role';

```

⊞ Action output

#	Time	Action	Message
✓ 43	13:15:40	REVOKE UPDATE ON Agri.soiltest FROM 'a...	0 row(s) affected

Vulnerable Query

In a login system, if user input is directly concatenated into SQL queries, the system becomes vulnerable to SQL Injection.

```

SELECT *
FROM farmers
WHERE farmer_name = 'Arun
Kumar' AND phone =
'9876543210';

```

```

48 SELECT *
49 FROM farmers
50 WHERE farmer_name = 'Arun Kumar' AND phone = '9876543210';

```

⊞ Result grid

farmer_id	farmer_name	phone	region	registration_date
12	Arun Kumar	9876543210	Tiruppur	2024-11-20

Secure Fix (Parameterized Query)

Instead of directly concatenating user input:

```
SELECT *  
FROM farmers WHERE farmer_name = :farmer_name AND phone = :phone;
```

Using bind variables:

```
VARIABLE  
fname  
VARCHAR2(50);  
VARIABLE  
fphone  
VARCHAR2(20);
```

```
EXEC :fname :=  
'Arun Kumar';  
EXEC :fphone :=  
'9876543210';
```

```
SELECT *  
FROM farmers  
WHERE  
farmer_name =  
:fname AND  
phone = :fphone;
```

Now the database treats input as data rather than executable SQL code. This prevents SQL Injection attacks and ensures secure access to the Agri Management System.

```
51 VARIABLE fname VARCHAR2(50);  
52 VARIABLE fphone VARCHAR2(20);  
  
53 EXEC :fname := 'Arun Kumar';  
54 EXEC :fphone := '9876543210';  
  
55 SELECT * FROM farmers  
56 WHERE farmer_name = :fname AND phone = :fphone;
```

Action output

#	Time	Action	Message
✓ 58	14:02:11	VARIABLE fname VARCHAR2(50)	Variable declared
✓ 59	14:02:11	VARIABLE fphone VARCHAR2(20)	Variable declared
✓ 60	14:02:12	EXEC :fname := 'Arun Kumar'	PL/SQL procedure...
✓ 61	14:02:12	EXEC :fphone := '9876543210'	PL/SQL procedure...
✓ 62	14:02:13	SELECT * FROM farmers WHERE farmer_name = :f...	1 row(s) returned